

3. Step 1 — Domaine & moteur de concurrence

Proposition de projet — Enterprise Inventory & Logistics Management System (EILMS)

Partie du document : Section 3.1 — Core Domain Modeling & Concurrency Engine

Objectif : poser le modèle orienté objet de base ET un moteur de traitement de commandes asynchrone, entièrement en mémoire (sans base de données à ce stade).

Les 3 instructions du Step 1

- 1. Interfaces et classes abstraites :** `InventoryItem`, `OrderProcessor` (interfaces) et `AbstractWarehouseItem` (classe abstraite). Une interface définit uniquement un contrat (« que doit savoir faire un item d'inventaire »), une classe abstraite peut en plus fournir du code partagé. Exemple : `AbstractWarehouseItem` pourrait implémenter la validation de quantité commune, tandis que des sous-classes comme `PerishableItem` ajoutent une date de péremption.
- 2. Streams fonctionnels :** utiliser l'API Stream de Java pour traiter des lots d'items — filtrer les stocks bas, formater des logs d'audit. Exemple : `items.stream().filter(i -> i.getQuantity() < 10).forEach(i -> log(i))` pour générer une alerte de rupture de stock.
- 3. Simulateur multithread :** un moteur de traitement de commandes utilisant `ExecutorService` (un pool de threads géré) et des files d'attente thread-safe (comme `BlockingQueue`). Exemple : plusieurs threads « consomment » des commandes dans la file en parallèle, sans se marcher dessus, simulant plusieurs employés qui traitent des commandes en même temps.

Pourquoi cette étape avant tout le reste

C'est exactement la logique déjà vue dans le document « Step 1 : Domain Model & In-Memory Data Layer » : construire un domaine solide et testable en mémoire avant d'ajouter la complexité d'une base de données ou d'un réseau. Ici, on ajoute en plus la dimension concurrence, absente du premier Step déjà traité.